For each Euler problem, we build a Father class

for each specific problem, we define Subclass

Names: (e.g. Constant, isentropic, Lax, Sod...)

Smooth: Yes or No

⋮

And specific Methods:

initial_cons (which problem, Coordinate)

initial_gamma (↓)

exact_cons (↓ Coordinate, time)

Source_term (↓ ↓ ↓)

|

hes_Soure (↓)

To run each problem, we need have discretization of a PDE
(which decided by mesh)

Given a son class of a problem

Attribute:

DG_case
State

Attribute:

① which sub-problem

② final_time

③ mesh

④ options

⑤ initial:: State → vertices_coordinate_mat
Cell_moments_array
at time 0
DG_State

⑥ gamma_cell.

We initiate DG_case by given

[p: problem, final_time, mesh, init_vex_coords_mat, options];

default init_cons (point_coords, cell_index)
function as. Problems.initial_cons(p, point_coords)

Initial = State.init_State(vex_coords, mesh, option, ini_cons)

We explict try a simplest case

Constant case <: problem, Euler problem

name :: Constant

gamma = 1.4

smooth = true

domain_bounds_list = [0, 1, 0, 1]

boundaries [periodic, ~, ~, ~]

density = 1

V_x = 0.3

V_y = -0.2

Pressure = 1

Problem: initial_cons (: constantprob, coordinate)

~ initial_gamma (

~ exact_cons (↓ ↓ coordinate, time) = E.P2C(P.density ...)

---

we don't use Source_term has_source for constant case so we don't define it!

Make a case: (problem: constant();

nx = 3, ny = 3,

degree = 2,

of = T,

characteristic = T

positivity = T

CFL = 0.5

final_time = 0.02)